RotorSubword: Geometric Algebra-Aware Tokenization for Cross-Linguistic Parity

Abstract

We present RotorSubword, an experimental subword tokenizer that uses geometric algebra, specifically Clifford algebra Cl(3,0), to guide merge decisions. Standard BPE-style tokenizers learn merges from corpus frequency and can therefore encode languages unevenly when the training distribution is unbalanced. In our FLORES-101 evaluation, GPT-2 requires 2.18x as many tokens per whitespace-delimited word for Indonesian as English, 33.29x for Chinese, and 60.08x for Japanese. RotorSubword embeds characters as multivectors and scores candidate merges using a grade-aware rotor alignment term. On a 12-language FLORES-101 experiment with a 500-token RotorSubword vocabulary and seven Hugging Face tokenizer baselines, RotorSubword improves the min/max fertility parity score to 0.089, compared with 0.039 for the strongest baseline in this run. In a separate vocabulary-matched comparison against GPT-2, a 49,822-token RotorSubword tokenizer reaches parity of 0.049 versus GPT-2's 0.018, while reducing the Chinese/English, Japanese/English, and Thai/English ratios. The result is not a drop-in replacement for production tokenizers: it trades absolute compression efficiency for parity, uses much smaller training data than real tokenizers, and still needs larger-corpus and downstream evaluations before stronger claims are warranted.

1 Introduction

Tokenization is the process of breaking text into units consumed by language models. Methods such as Byte-Pair Encoding (BPE; Sennrich et al., 2016), WordPiece (Schuster and Nakajima, 2012), and Unigram tokenization (Kudo, 2018) optimize token inventories using statistical objectives over a training corpus. When that corpus is dominated by a subset of languages or scripts, the resulting vocabulary can be much more efficient for those languages than for others.

This imbalance has practical consequences. More tokens for the same text means higher inference cost, less effective context length, and potentially weaker downstream performance. Prior work has documented large tokenization gaps across languages and scripts (Ahia et al., 2023; Petrov et al., 2023).

RotorSubword explores a different merge prior. Rather than ranking candidate merges by raw frequency alone, it embeds characters and merged tokens as multivectors in Cl(3,0), computes a rotor-like relationship between adjacent tokens, and favors pairs whose relationship has a clean grade profile. The hypothesis is modest: a script-agnostic geometric prior can reduce cross-linguistic token-count disparities, even if it is not yet as compression-efficient as large production tokenizers.

Our contributions are:

1. A rotor-guided subword tokenizer that combines frequency and Cl(3,0) grade-aware alignment for merge decisions.

2. A differentiable Cl(3,0) geometric product implementation using a fixed structure tensor.

3. A multivector tokenizer prototype with learnable token embeddings and rotor-consistency, grade-prediction, and reconstruction losses.

4. An empirical comparison on FLORES-101 across 12 languages and seven baseline tokenizers, plus a vocabulary-matched comparison against GPT-2.

2 Background

2.1 Subword Tokenization

BPE iteratively merges frequent adjacent token pairs. WordPiece and Unigram tokenizers use different selection criteria, but they share the same broad dependency on the training distribution. This makes tokenization efficiency sensitive to corpus composition, script, normalization choices, and vocabulary size.

2.2 Geometric Algebra

Clifford algebra Cl(3,0) represents geometric quantities in three Euclidean dimensions. Its elements are multivectors:

- grade 0: scalars
- grade 1: vectors
- grade 2: bivectors
- grade 3: trivectors or pseudoscalars

The geometric product combines inner-product-like and exterior-product-like structure:

$$ab = a \cdot b + a \wedge b$$

Rotors represent rotations in planes described by bivectors. RotorSubword uses this machinery as a merge-scoring prior: adjacent tokens are preferred when their multivector relationship yields a strong bivector component and limited high-grade noise.

2.3 Tokenization Bias

Tokenization bias has been measured across many modern tokenizers. In the experiments reported here, GPT-2 is strongly English-centric, while multilingual tokenizers such as XLM-R, mGPT, BLOOM, Qwen-2.5, and multilingual BERT reduce but do not remove large disparities for CJK and Thai under our fertility metric.

3 Method

3.1 Character Embeddings as Multivectors

Each character `c` is embedded as an 8-dimensional Cl(3,0) multivector with basis

$$\{1, e_1, e_2, e_3, e_{12}, e_{13}, e_{23}, e_{123}\}.$$

The current implementation uses deterministic Unicode codepoint hashing for the vector components:

$$v_1 = \sin(0.1 \cdot \mathrm{ord}(c))$$

$$v_2 = \cos(0.137 \cdot \mathrm{ord}(c))$$

$$v_3 = \sin(0.271 \cdot \mathrm{ord}(c) + 1).$$

For Chinese characters, two bivector components are added:

$$b_1 = 0.5 \sin(0.19 \cdot \mathrm{ord}(c))$$

$$b_2 = 0.5 \cos(0.23 \cdot \mathrm{ord}(c)).$$

The vector part is normalized after initialization. This embedding is intentionally simple; it is a deterministic geometric feature map, not a learned linguistic representation.

3.2 Geometric Product

The Cl(3,0) geometric product is implemented as a bilinear operation over a fixed structure tensor:

$$\mathrm{GP}(a, b)_k = \sum_{i,j} M_{kij} a_i b_j.$$

In code, this is computed with `torch.einsum('kij,bij->bk', gp_mat, outer)`, so gradients can flow through multivector operands. The multiplication table is checked against canonical identities such as:

- $e_1 e_1 = 1$
- $e_1 e_2 = e_{12}$
- $e_2 e_1 = -e_{12}$
- $e_{12} e_{12} = -1$
- $e_{12} e_{13} = -e_{23}$
- $e_{123} e_{123} = -1$

3.3 Rotor-Guided Merge Scoring

For a candidate bigram `(a, b)` with count `n`, RotorSubword computes an approximate rotor from the bivector part of `GP(M(a), M(b))`. It then extracts grade norms `(s, v, b, t)` for scalar, vector, bivector, and trivector components.

The current implementation uses:

$$\mathrm{alignment}(a, b) = \max\left(\frac{2b + 0.5v - 0.2t}{s + v + b + t + \epsilon}, 0.01\right).$$

The bigram score is a hybrid of geometric alignment and frequency:

$$\mathrm{score}(a, b) = (1 - \lambda)\,\mathrm{alignment}(a, b)\sqrt{n} + \lambda n.$$

The FLORES-101 results below use `freq_weight = 0.0`, so scores are geometric-alignment-weighted square-root frequency rather than raw BPE frequency.

3.4 Iterative Merge Training

Training follows a BPE-like loop:

1. Initialize the vocabulary from observed characters.
2. Count adjacent token pairs in the current corpus.
3. Score each pair with the hybrid rotor/frequency score.
4. Select the highest-scoring pair, or a batch of non-conflicting high-scoring pairs when `batch_size > 1`.
5. Merge selected pairs throughout the corpus.
6. Assign the merged token a composed multivector:

$$M(ab) = \text{normalize}\left(\frac{M(a) + M(b)}{2} + 0.3\,\text{GP}(M(a), M(b))\right).$$

The 500-vocabulary experiment uses `batch_size = 1`, matching standard iterative BPE behavior. The matched-vocabulary experiment uses batch merging for tractable training time.

3.5 Learnable Multivector Tokenizer

The repository also includes `TokenMultivectorTokenizer`, a prototype where each token has a learnable multivector parameter. It combines three training signals:

- **Rotor consistency**: related tokens should produce cleaner rotor relationships than unrelated tokens.
- **Grade-wise prediction**: selected grades are encouraged to encode token-level properties such as importance or relational structure.
- **Reconstruction**: merged-token multivectors should be reconstructible from their components using the same composition rule as above.

These objectives are preliminary and are not the source of the main FLORES-101 table.

4 Experiments

4.1 Setup

Dataset: FLORES-101 devtest.

Languages: Arabic, Chinese, English, Hindi, Indonesian, Japanese, Javanese, Korean, Malay, Tagalog, Thai, and Vietnamese.

Small-vocabulary RotorSubword: 100 sentences per language, vocabulary size 500.

Matched-vocabulary RotorSubword: 50 sentences per language, vocabulary size approximately 50,000 (48,073 merges), using batch merging (batch size 100) for tractable training time.

RotorSubword evaluation: held-out sentences 100–199 (small vocab) or 200–299 (matched vocab) for each language.

Baselines: GPT-2 (50,257 vocab), Qwen-2.5 (151,643), XLM-R (250,002), mGPT (100,000), BLOOM (250,680), Mistral (32,000), and BERT-multilingual (119,547). Baselines use their released vocabularies. The matched-vocabulary comparison is against GPT-2 only.

Metrics:

- **Fertility**: tokens per whitespace-delimited word.
- **Ratio vs English**: fertility of a language divided by English fertility.
- **Parity score**: min(fertility) / max(fertility) across languages, where higher is better and 1.0 is perfect parity.

The fertility metric is imperfect for languages without whitespace word segmentation, including Chinese, Japanese, and Thai. The resulting absolute fertility values should not be interpreted as linguistically exact word-level costs. They are still useful as a controlled tokenizer comparison because every tokenizer is evaluated against the same text and denominator.

4.2 Small-Vocabulary Results

Table 1 reports ratio vs English for selected non-English languages. Parity is computed over all 12 evaluated languages.

Table 1: Ratio vs English.

Tokenizer	ar	hi	id	ja	ko	ms	th	tl	vi	zh	Parity
RotorSubword	**0.97**	**0.84**	1.18	**8.32**	**0.74**	1.20	**4.62**	**1.02**	0.75	**2.97**	**0.089**
XLM-R	1.31	1.07	**1.04**	22.57	1.65	**1.03**	5.50	1.17	**0.84**	10.16	0.037
BLOOM	1.28	1.09	1.06	35.81	3.90	1.15	23.26	1.51	0.90	9.68	0.025
Qwen-2.5	1.82	3.77	1.70	28.42	2.31	1.74	12.94	1.66	1.02	10.62	0.035
mGPT	1.56	2.68	1.34	24.40	2.08	1.34	12.90	1.49	0.95	12.71	0.039
BERT-multilingual	1.70	1.51	1.26	31.35	1.98	1.26	13.79	1.40	0.92	14.75	0.029
Mistral	3.87	3.89	2.03	43.39	3.48	2.06	21.47	1.71	2.08	17.05	0.023
GPT-2	4.86	6.33	2.18	60.08	7.13	2.22	45.71	1.86	3.22	33.29	0.017

RotorSubword has the best parity score in this run: 0.089 versus 0.039 for mGPT, the strongest baseline by this metric. It also produces the lowest tested ratios for Chinese, Japanese, Korean, Thai, Tagalog, Arabic, and Hindi. XLM-R remains stronger on Indonesian, Malay, and Vietnamese.

Table 2: Raw fertility for RotorSubword and representative baselines.

Tokenizer	ar	en	hi	id	ja	jv	ko	ms	th	tl	vi	zh
RotorSubword	5.86	6.04	5.09	7.15	50.25	6.88	4.45	7.26	27.94	6.16	4.52	17.95
GPT-2	6.00	1.24	7.82	2.69	74.20	2.62	8.81	2.74	56.45	2.30	3.98	41.12
Qwen-2.5	2.29	1.26	4.76	2.15	35.84	2.37	2.92	2.20	16.32	2.10	1.29	13.39
XLM-R	1.83	1.40	1.49	1.45	31.59	1.77	2.30	1.44	7.70	1.64	1.18	14.22

The raw table shows the main tradeoff at small vocabulary. RotorSubword improves parity by raising English fertility substantially while reducing the largest non-English ratios. This is a fairness-oriented tokenizer, not an efficiency-oriented tokenizer.

4.3 Matched-Vocabulary Comparison

The small-vocabulary results above use a 500-token RotorSubword vocabulary against baselines with 32k–250k tokens. To address this mismatch, Table 3 presents a GPT-2-matched experiment where RotorSubword is trained to 49,822 total tokens (48,073 learned merges) on 50 sentences per

Table 3: Matched-vocabulary comparison: RotorSubword approximately 50k vs GPT-2 50k.

Language	RotorSubword	R/EN	GPT-2	R/EN
Arabic	2.91	1.14	5.99	4.67
English	2.55	1.00	1.28	1.00
Hindi	2.39	0.94	7.78	6.06
Indonesian	2.72	1.07	2.66	2.07
Japanese	38.78	15.21	72.05	56.16
Javanese	2.77	1.09	2.56	1.99
Korean	3.01	1.18	8.94	6.97
Malay	2.71	1.06	2.69	2.10
Thai	10.29	4.03	41.75	32.54
Tagalog	2.31	0.90	2.24	1.75
Vietnamese	1.89	0.74	3.84	2.99
Chinese	13.60	5.33	29.27	22.81
Parity		**0.049**		0.018

language. This closely matches GPT-2's vocabulary size of 50,257, but not GPT-2's training data scale.

At matched vocabulary size, RotorSubword achieves parity of 0.049, about **2.7x better** than GPT-2's 0.018. The ratio-vs-English gap is smaller for RotorSubword across every non-English language in this comparison:

- **Chinese**: 5.33x vs 22.81x (4.3x better)
- **Japanese**: 15.21x vs 56.16x (3.7x better)
- **Thai**: 4.03x vs 32.54x (8x better)
- **Indonesian**: 1.07x vs 2.07x (nearly equal to English under GA)
- **Hindi**: 0.94x (lower fertility than English under RotorSubword in this run)

English fertility is higher under RotorSubword (2.55 vs 1.28), confirming that the parity-efficiency tradeoff persists at matched vocabulary sizes. However, the gap narrows substantially compared to the small-vocabulary experiment (2.55 vs 6.04).

4.4 Characters-Per-Token: A Script-Fair Metric

Fertility (tokens per whitespace-delimited word) disadvantages scripts that do not use whitespace to delimit words: Chinese, Japanese, and Thai inflate the denominator because they have zero or few whitespace-delimited "words." Table 4 recomputes parity using characters per token (CPT), which avoids this bias.

Under CPT parity, RotorSubword achieves 0.443, about 4.2x better than GPT-2's 0.105. Latin-script and abjad languages approach English parity under RotorSubword (Arabic 0.84, Hindi 0.89, Indonesian 1.11), while CJK and Thai remain lower but are dramatically closer than under GPT-2 (Chinese 0.50 vs 0.12, Japanese 0.58 vs 0.16).

The CPT metric reveals that GPT-2's apparent efficiency for CJK languages under whitespace fertility can be misleading: those "words" are often entire sentences because whitespace is absent.

Table 4: Characters-per-token and CPT ratio vs English, matched 50k vocabulary.

Language	GA CPT	GA R/EN	GPT-2 CPT	GPT-2 R/EN
Arabic	2.008	0.840	0.976	0.206
English	2.391	1.000	4.744	1.000
Hindi	2.123	0.888	0.653	0.138
Indonesian	2.647	1.107	2.706	0.570
Japanese	1.375	0.575	0.740	0.156
Javanese	2.485	1.039	2.685	0.566
Korean	1.495	0.625	0.498	0.105
Malay	2.687	1.124	2.696	0.568
Thai	2.169	0.907	0.539	0.114
Tagalog	2.675	1.119	2.750	0.580
Vietnamese	2.457	1.028	1.212	0.255
Chinese	1.189	0.497	0.552	0.116
CPT Parity		**0.443**		0.105

CPT corrects this by measuring how many characters each token covers, regardless of word segmentation.

4.5 Observations

Parity improves at both vocabulary sizes. At 500 vocab, parity is 0.089 vs 0.039 for mGPT (next best). At matched 50k vocab, parity is 0.049 vs 0.018 for GPT-2, a 2.7x improvement. The gap closes somewhat at larger vocabulary because more merges give both tokenizers more capacity, but RotorSubword retains a substantial advantage.

The fairness-efficiency tradeoff narrows at scale. English fertility drops from 6.04 at 500 vocab to 2.55 at 50k vocab, compared to GPT-2's 1.28. At 50k, RotorSubword's English fertility is within 2x of GPT-2's, while achieving 2.7x better parity.

CJK and Thai gaps shrink but remain large. Chinese improves from 22.81x (GPT-2) to 5.33x (RotorSubword 50k). Japanese improves from 56.16x to 15.21x. Thai improves from 32.54x to 4.03x. Under CPT parity, the picture is even more favorable: RotorSubword achieves 0.443 vs GPT-2's 0.105 (4.2x better), because CPT accounts for the fact that CJK characters carry more semantic content per character.

Latin-script ratios are close to English at matched vocabulary. Indonesian (1.07x), Malay (1.06x), and Javanese (1.09x) are close to English. Arabic (1.14x) and Hindi (0.94x) are also close. Vietnamese (0.74x) has lower fertility than English in this run, which should be interpreted cautiously because the metric is corpus- and tokenizer-dependent.

4.6 Ablation Snapshot

Earlier three-language experiments on English, Indonesian, and Chinese suggest that the corrected geometric product, grade-aware scoring, and richer Chinese character embeddings each improve parity.

This table should be read as a development trace rather than a controlled final ablation. Some rows differ in corpus size, merge procedure, and evaluation setup.

Table 5: Development ablation snapshot.

Configuration	ID/EN	ZH/EN	Parity
GPT-2 baseline	2.13	21.72	0.026
Character-level, no merging	about 1.0	about 1.0	about 1.0
Rotor-guided, corrected GP	1.16	2.27	0.80
+ Grade-aware scoring	1.10	2.20	higher
+ Rich Chinese embeddings	1.15	2.20	0.80
+ Iterative BPE-style merging	0.97	3.55	0.168

4.7 Geometric Product Correctness

The Cl(3,0) multiplication table is central to the approach. During development, sign errors in the initial table changed several identities, including bivector products and the pseudoscalar square. The current implementation verifies representative canonical identities in `gatoken/clifford.py`.

5 Discussion

5.1 The Parity-Efficiency Tradeoff

RotorSubword demonstrates that a geometric merge prior can improve cross-language fertility parity, but the improvement comes with worse absolute compression. This matters: a production model using RotorSubword as-is would spend more tokens on English and many Latin-script languages than standard tokenizers.

The approach may still be useful in settings where equalized token budgets matter more than raw compression, or as a component in a hybrid tokenizer that combines production-grade compression with a parity-aware penalty.

5.2 Why a Geometric Prior Helps

BPE rewards pairs that are frequent in the training corpus. RotorSubword instead rewards pairs whose embeddings have a cleaner grade structure under the geometric product. Because the same algebra is used for all scripts, the scoring rule is less directly tied to the frequency profile of any one script. This is the intended mechanism behind the parity improvement.

5.3 Remaining Script Challenges

Chinese, Japanese, and Thai remain difficult under this metric. The whitespace denominator inflates fertility for scripts that do not delimit words with spaces, and the tokenizer itself still begins from characters. Better evaluation should include script-appropriate segmentation, bytes or characters per token, and downstream model performance.

5.4 Differentiable GA for Learning

The differentiable geometric product makes it possible to learn multivector embeddings end to end. The current learned-token experiments are preliminary; they show that gradients propagate through the algebraic operations, but they do not yet establish downstream benefits.

5.5 Limitations

1. **Training data scale**: RotorSubword is trained on 50–100 sentences per language; production tokenizers use billions of tokens. The matched-vocabulary results reduce the vocabulary-size mismatch but not the training-data mismatch.

2. **Absolute efficiency**: RotorSubword uses more English tokens than production tokenizers (2.55 vs 1.28 tokens/word at matched 50k vocab). The fairness-efficiency tradeoff is real.

3. **Metric limitations**: whitespace fertility is not linguistically adequate for all scripts. Characters-per-token or bytes-per-token would be more appropriate for CJK and Thai.

4. **No downstream evaluation**: the paper measures token counts, not model quality. Whether fairer tokenization leads to better cross-lingual performance is an open question.

5. **Simple embeddings**: deterministic codepoint hashing is a weak proxy for linguistic structure. Learned multivector embeddings could improve results.

6 Related Work

- **Tokenization fairness and tokenizer evaluation**: Ahia et al. (2023) document API cost disparities caused by tokenizer-dependent token counts, while Petrov et al. (2023) show that tokenization length disparities persist even for multilingual tokenizers. Ali et al. (2024) evaluate tokenizer choice in LLM training and caution that fertility and parity are not always reliable proxies for downstream quality. Zouhar et al. (2023) analyze tokenizer quality through an information-theoretic channel-efficiency lens.

- **Subword tokenization**: Sennrich et al. (2016) introduce BPE for neural machine translation. Schuster and Nakajima (2012) describe WordPiece-style tokenization in Japanese and Korean voice search. Kudo (2018) introduces subword regularization and the Unigram language model tokenizer, while Kudo and Richardson (2018) introduce SentencePiece as a language-independent tokenizer/detokenizer that can train from raw text.

- **Token-free and learned tokenization models**: CANINE (Clark et al., 2022), ByT5 (Xue et al., 2022), and Charformer (Tay et al., 2022) avoid or soften fixed subword vocabularies by operating on characters, bytes, or learned character-block representations. These approaches address similar brittleness and multilingual coverage concerns from the model architecture side, whereas RotorSubword keeps a tokenizer interface and changes the merge prior.

- **Multilingual evaluation**: Goyal et al. (2022) introduce FLORES-101 as a multilingual evaluation benchmark with aligned sentences across 101 languages.

- **Geometric algebra in neural networks**: Brandstetter et al. (2023), Ruhe et al. (2023a, 2023b), and Brehmer et al. (2023) show how Clifford/geometric algebra can support equivariant or geometry-aware neural architectures. RotorSubword uses the same algebraic toolkit for tokenization rather than for downstream neural layers.

7 Conclusion

RotorSubword is an experimental geometric algebra-aware tokenizer that uses Cl(3,0) multivectors, rotor-inspired alignment, and grade-aware scoring to reduce cross-linguistic token-count disparities.

On the FLORES-101 benchmark across 12 languages, it achieves fertility parity of 0.089 at 500 vocab, about 2.3x better than the strongest baseline in that run. In the separate GPT-2-matched 50k comparison, it achieves fertility parity of 0.049 vs GPT-2's 0.018, about a 2.7x improvement. Under the script-fair characters-per-token metric, it achieves CPT parity of 0.443 at matched 50k vocab, 4.2x better than GPT-2's 0.105. At matched vocabulary, Chinese tokenization is 4.3x fairer than GPT-2 (5.33x vs 22.81x), Japanese is 3.7x fairer, and Thai is 8x fairer. The fairness-efficiency tradeoff narrows at scale: English fertility drops from 6.04 (500 vocab) to 2.55 (50k vocab), within 2x of GPT-2. Future work should evaluate larger training corpora, hybrid frequency-geometric objectives, and downstream language-model performance.

References

[1] Orevaoghene Ahia, Sachin Kumar, Hila Gonen, Jungo Kasai, David Mortensen, Noah Smith, and Yulia Tsvetkov. Do all languages cost the same? tokenization in the era of commercial language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 9904–9923, Singapore, 2023. Association for Computational Linguistics.

[2] Mehdi Ali, Michael Fromm, Klaudia Thellmann, Richard Rutmann, Max Lübbering, Johannes Leveling, Katrin Klug, Jan Ebert, Niclas Doll, Jasper Buschhoff, Charvi Jain, Alexander Weber, Lena Jurkschat, Hammam Abdelwahab, Chelsea John, Pedro Ortiz Suarez, Malte Ostendorff, Samuel Weinbach, Rafet Sifa, Stefan Kesselheim, and Nicolas Flores-Herr. Tokenizer choice for LLM training: Negligible or crucial? In *Findings of the Association for Computational Linguistics: NAACL 2024*, pages 3907–3924, Mexico City, Mexico, 2024. Association for Computational Linguistics.

[3] Johannes Brandstetter, Rianne van den Berg, Max Welling, and Jayesh K. Gupta. Clifford neural layers for PDE modeling. In *International Conference on Learning Representations*, 2023.

[4] Johann Brehmer, Pim de Haan, Sönke Behrends, and Taco Cohen. Geometric algebra transformer. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

[5] Jonathan H. Clark, Dan Garrette, Iulia Turc, and John Wieting. CANINE: Pre-training an efficient tokenization-free encoder for language representation. *Transactions of the Association for Computational Linguistics*, 10:73–91, 2022.

[6] Naman Goyal, Cynthia Gao, Vishrav Chaudhary, Peng-Jen Chen, Guillaume Wenzek, Da Ju, Sanjana Krishnan, Marc'Aurelio Ranzato, Francisco Guzmán, and Angela Fan. The FLORES-101 evaluation benchmark for low-resource and multilingual machine translation. *Transactions of the Association for Computational Linguistics*, 10:522–538, 2022.

[7] Taku Kudo. Subword regularization: Improving neural network translation models with multiple subword candidates. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 66–75, Melbourne, Australia, 2018. Association for Computational Linguistics.

[8] Taku Kudo and John Richardson. SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 66–71, Brussels, Belgium, 2018. Association for Computational Linguistics.

[9] Aleksandar Petrov, Emanuele La Malfa, Philip H. S. Torr, and Adel Bibi. Language model tokenizers introduce unfairness between languages. In *Advances in Neural Information Processing Systems*, 2023. arXiv:2305.15425.

[10] David Ruhe, Johannes Brandstetter, and Patrick Forré. Clifford group equivariant neural networks. In *Advances in Neural Information Processing Systems*, 2023.

[11] David Ruhe, Jayesh K. Gupta, Steven de Keninck, Max Welling, and Johannes Brandstetter. Geometric clifford algebra networks, 2023.

[12] Mike Schuster and Kaisuke Nakajima. Japanese and korean voice search. In *2012 IEEE International Conference on Acoustics, Speech and Signal Processing*, pages 5149–5152, 2012.

[13] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1715–1725, Berlin, Germany, 2016. Association for Computational Linguistics.

[14] Yi Tay, Vinh Q. Tran, Sebastian Ruder, Jai Gupta, Hyung Won Chung, Dara Bahri, Zhen Qin, Simon Baumgartner, Cong Yu, and Donald Metzler. Charformer: Fast character transformers via gradient-based subword tokenization. In *International Conference on Learning Representations*, 2022.

[15] Linting Xue, Aditya Barua, Noah Constant, Rami Al-Rfou, Sharan Narang, Mihir Kale, Adam Roberts, and Colin Raffel. ByT5: Towards a token-free future with pre-trained byte-to-byte models. *Transactions of the Association for Computational Linguistics*, 10:291–306, 2022.

[16] Vilém Zouhar, Clara Meister, Juan Gastaldi, Li Du, Mrinmaya Sachan, and Ryan Cotterell. Tokenization and the noiseless channel. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 5184–5207, Toronto, Canada, 2023. Association for Computational Linguistics.